dSABRE: A SABRE-Style Router for Multi-Core Distributed Quantum Computers

Sanjiang Li *

***Abstract*—A distributed quantum computer (DQC) runs a circuit across several small cores linked by entanglement. A gate whose operands sit in different cores cannot be executed directly: either the operands are brought into the same core, or the gate is executed remotely. Both primitives consume Einstein–Podolsky–Rosen (EPR) pairs—orders of magnitude costlier than a local gate, and so the quantity a distributed router must minimise. Routing at this scale needs a fast heuristic rather than an exact method. On a single chip that role is filled by SABRE, the default layout and routing method in Qiskit; its front-layer-plus-lookahead score is what we carry over to the inter-core setting. We present dSABRE, a SABRE-style distributed router that moves operands rather than gates, built around three choices: a teleport score whose dominant term penalises a candidate in proportion to how full its landing core already is; a lookahead set built in BFS layers over the circuit's directed acyclic graph (DAG), so a gate's influence decays with its depth; and pre-emptive eviction of idle qubits from cores that upcoming traffic will saturate, backed by a checkpoint–rollback escape with a bounded stall cost. Across 21 MQT-Bench circuits at 25–64 logical qubits dSABRE cuts geometric-mean EPR consumption by 41–46% against TeleSABRE, and across the full evaluation routes four circuits on which TeleSABRE fails to converge. The margin is 48% on a non-grid architecture of IBM heavy-hex cores with no parameter retuned, and a quantum Fourier transform (QFT) sweep to 360 qubits shows the router remains practical at that size. Code and online appendices are at https://github.com/ebony72/dsabre.**

***Index Terms*—distributed quantum computing, circuit routing, qubit teleportation, SABRE, EPR minimisation, qubit mapping, lookahead heuristic**

I. Introduction

The near-term path to fault-tolerant quantum computing requires processors with thousands of high-fidelity qubits. No single monolithic chip can economically deliver this scale today. Instead, the field is converging on *distributed quantum computers* (DQC) [1]: multiple small quantum processing units (QPUs, also called cores) interconnected by photonic or microwave links that support remote entanglement generation.

Executing a quantum circuit on a DQC requires a *distributed compilation* pipeline that, like its monolithic counterpart, decomposes into three problem classes [2]. *(i) Qubit allocation* statically assigns logical qubits to cores, typically by hypergraph partitioning [3], [4]. *(ii) Communication-primitive selection* decides how each non-local two-qubit gate is realised—by bringing both qubits to the same core, or by executing the gate remotely [5], [6] (Section II-B describes both primitives and their costs). *(iii) Routing and scheduling* orders the resulting intra-core SWAPs, teleports, and entanglement-generation requests in time, subject to communication-port capacity and finite entanglement rates [7], [8].

All three stages are judged against the same currency. Every communication primitive consumes one or more EPR pairs, and EPR generation is slow (microseconds to milliseconds), noisy, and rate-limited in current hardware [9], [10]—orders of magnitude more expensive than a local gate. EPR consumption is therefore the dominant objective of distributed compilation, and the one this paper minimises.

dSABRE addresses the routing component of stage (iii) — ordering intra-core SWAPs and teleports — and leaves scheduling (EPR-generation timing, port reservation) to downstream passes; within stage (ii) it uses only state teleportation, and it is complementary to the static-allocation methods of stage (i).

Routing runs inside a compiler, on circuits of the size a DQC exists to execute—this paper routes up to 360 logical qubits on 486 physical ones—so the routing rule must be a fast heuristic evaluated per candidate move rather than a global optimisation. On a single chip that role is filled by SABRE [11], which Qiskit [12] selects by default for both layout and routing and which subsequent work has refined rather than replaced [13]. Its objective is local and cheap: score a candidate move by the change it induces in the distance between the operands of the front-layer gates, plus a lookahead term over a bounded window of upcoming gates. That objective is what carries over to the multi-core setting, and the distributed routers closest to this work are built from it [14], [15]. dSABRE is too, and the question this paper answers is what that objective must become when one chip becomes several.

The most recent and best-performing of these is TeleSABRE [15], which uses a Dijkstra computation on a contracted communication graph to estimate inter-core routing cost. We observe one substantive limitation: congestion is handled defensively. TeleSABRE discourages landing in saturated cores via a full-core penalty and recovers from deadlock via a safety-valve mode that drops most of the lookahead, but it never proactively *evicts* idle qubits out of crowded cores ahead of demand. On dense or large instances this recovery either burns additional EPR pairs or fails to converge altogether.

Contributions.

- **A five-term gate-centric teleportation score.** LightSABRE [13] scores a SWAP as the front-layer distance change Δ_F plus a decay-weighted lookahead $\tilde{\Delta}_E$, both in Δ-form (new minus old distance, so lower is better) on gates sharing a qubit with the move. dSABRE carries the same two Δ-form terms into the

S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).
*ORCID: 0000-0002-3332-2546.

inter-core scorer and adds one term for each respect in which a teleport is unlike a SWAP: a staging cost d_{prep}, because a teleport can fire only at a communication port; a *core-capacity penalty* c_{cap}, because a core has finitely many slots and each teleport consumes one; and a hop-direction reward g_{hop}, because the qubit lands at a port that is in general not where its next hop begins (Section III-D). Ablation (Section IV-E) shows the capacity term carries the score: removing it inflates geometric-mean EPR by $+60.8\%$ on the 25-qubit suite and $+53.5\%$ on the 64-qubit suite, against $+11.3$–26.5% for the lookahead and under 9% for any other single change.

- **BFS-layer inter-core extended set.** The inter-core scorer builds its lookahead set layer by layer over the remaining DAG so that gates sharing a wire with the front are captured before unrelated wires displace them, with each gate's BFS depth driving the decay exponent $\gamma^{\text{dep}(g)}$. TeleSABRE also slices in layers, but emits within a layer in arrival order and weights the g-th emitted gate by $(1 + g/10)$, which grows with emission index instead of decaying with depth. Proposition 1 shows layer expansion is what fills the size-L window in depth order, which topological filling cannot do. Substituting a topological-order extended set increases geometric-mean EPR by 6.8–10.9% across the three suites, with the gain growing with circuit size (Section IV-E).
- **Proactive congestion relief and a checkpoint–rollback escape.** dSABRE maintains an explicit demand vector over cores and proactively teleports idle qubits away from high-demand, low-capacity cores before bottlenecks form, scoring relief moves within the same teleportation objective; a checkpoint–rollback escape forces shortest-path progress on a stuck gate. Neither lowers EPR consumption on the geometric mean—relief is inert at 25 qubits, and disabling it *lowers* 64-qubit gmean EPR by 8.7% (Section IV-E), because the per-core initial layout now gives every core reserved escape capacity and so preempts most of the congestion relief was built to clear. Their value is completion: dSABRE routes the 64-qubit Random and QAOA circuits, the 200-qubit QFT, and QNN on the heavy-hex architecture, on all of which TeleSABRE fails to converge. Proposition 2 bounds the cost of escaping a stall at twice the core-graph diameter in EPR pairs under a stated capacity condition, where a fixed safety-valve cap offers no progress argument.
- **Comprehensive empirical validation.** Across MQT-Bench suites at 25, 36, and 64 qubits on multi-core grid architectures, dSABRE reduces geometric-mean EPR consumption by 41–46% over TeleSABRE [15], the one baseline that shares dSABRE's cost model, physical architecture, and intra-core SWAP accounting. The margin is 48% on a ring of IBM heavy-hex cores with no hyperparameter retuned. `pytket-dqc` [4] is evaluated in full but reported with its own accounting rather than as a headline figure, since its e-bit counts charge no intra-core routing, ignore how many physical links join two cores, assume unbounded entanglement lifetime, and group gates (Section IV-A). dSABRE uses a `SabreLayout`-based initial layout with per-core communication-slot reservation. Ablation studies, cost-ratio sweeps, and a scalability sweep up to 360-qubit QFT locate the gains in the scoring design.

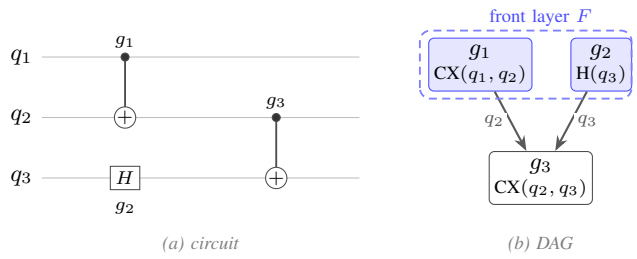


Fig. 1. DAG for a three-gate circuit on qubits q_1, q_2, q_3. CX(q_1, q_2) and H(q_3) share no qubit and have no incoming edges, so both appear in the initial front layer $F = \{g_1, g_2\}$ and may execute in parallel. CX(q_2, q_3) depends on both via qubits q_2 and q_3; it enters F only after g_1 and g_2 have been executed. Circuit depth is 2 (two sequential layers).

Paper organisation. Section II fixes notation and recaps the SABRE heuristic. Section III presents dSABRE: the routing loop, the intra-core and inter-core scoring rules, the BFS-layer extended set, proactive congestion relief, the checkpoint–rollback escape, and a complexity analysis. Section IV evaluates dSABRE on the 25q, 36q, and 64q MQT-Bench suites against TeleSABRE and `pytket-dqc`, ablates the design choices, characterises the initial-layout pipeline, reports compile time, and runs a 100–360q QFT scalability sweep. Section V positions dSABRE against related work; Section VI outlines directions for further work; Section VII concludes.

II. Background

A. Quantum Circuits

A quantum circuit acts on n *qubits* through a sequence of *gates*. *One-qubit (1Q) gates* (Hadamard, rotations, ...) act on a single qubit. *Two-qubit (2Q) gates* entangle pairs: the *CX* (CNOT) gate is the standard primitive and the building block for most multi-qubit operations. Together, CX and the full single-qubit unitary group $U(2)$ form a universal gate set [16]: every n-qubit unitary decomposes exactly into CX gates and single-qubit rotations. A finite basis such as $\{\text{CX}, H, T\}$ suffices for approximate universality and is what the benchmark circuits of Section IV-A are transpiled into. The *SWAP* gate exchanges the states of two adjacent qubits; it decomposes into three CX gates and is used by routing algorithms to migrate logical qubit states along the coupling graph.

A circuit is represented as a *directed acyclic graph* (DAG) in which each node is a gate and each directed edge records a data dependency (qubit produced by gate g_1 consumed by gate g_2). For every qubit q, each gate acting on q is connected to the *next* gate acting on q: the second cannot begin until the first has written its output on q. Figure 1 shows a small example, where *circuit depth* is the length of the longest path through the DAG.

Physical qubits are arranged in a *coupling graph* $G_{\text{phys}} = (P, E_{\text{phys}})$; a 2Q gate can execute only if its two operands

occupy adjacent vertices. A circuit is written in terms of *logical qubits* $\mathrm{q}_1, \ldots, \mathrm{q}_n$; a *layout* $\ell : \mathrm{q}_i \mapsto \mathrm{p}_p$ assigns each to a physical slot. *Qubit routing* inserts SWAP gates so that the two operands of every 2Q gate reach adjacent physical qubits before it executes.

B. Quantum Teleportation

Quantum teleportation [5] transfers the state of a qubit from a sender to a receiver—in the DQC setting, from one core to another over an inter-core link—using a pre-shared *EPR pair* (a maximally entangled two-qubit state) and two classical bits. The source qubit's state is destroyed at the sender and reconstructed at the receiver; no physical qubit travels across the link. Each EPR pair is consumed upon use, so the *EPR count* is the primary inter-core routing cost in distributed quantum compilation. Throughout this paper, "EPR" as a unit (e.g. "10 EPRs") denotes EPR-pair count; "EPR pair" is used when emphasising the physical entangled state itself.

Two inter-core teleportation variants arise in DQC compilation. *Teledata* moves a logical qubit to a new core (one EPR pair consumed), after which the qubit participates in local gates there. *Telegate* [6] executes a non-local 2Q gate directly across two cores via a shared EPR pair, without physically relocating either qubit. dSABRE uses teledata exclusively, and because TeleSABRE exploits both variants this deserves a justification rather than a statement. The two primitives have different cost structures. Teledata pays once per relocation and then makes *every* subsequent gate on that qubit local, so its cost amortises over a qubit's whole residence in a core; telegate pays once per non-local gate, or once per group of gates if consecutive non-local gates on a shared control are amortised onto one entangled state. Which is cheaper is circuit-dependent, and Section IV-A makes the accounting explicit. They differ, however, in one respect that is not circuit-dependent: a teledata EPR pair is consumed instantaneously by its teleport, whereas an amortised telegate requires a link to stay entangled for the span of the gate group it serves. Teledata therefore delivers its cost under any entanglement lifetime, while gate-teleportation counts are contingent on the hardware sustaining entanglement—an assumption we test directly in Section IV-B. On fairness, the restriction cuts against dSABRE, not for it: it competes with a strict subset of TeleSABRE's primitives and still wins on the large majority of circuits, so the comparison understates rather than flatters the result. Adding telegate candidates to the scorer is a genuine extension and we scope it as future work (Section VI).

C. Distributed Quantum Architecture

We model a DQC architecture $\mathcal{A}$ abstractly as a weighted graph $G_{\mathcal{A}} = (V, E, w)$ whose vertices are physical qubits. The vertex set is partitioned into K *cores* $V = V_1 \sqcup \cdots \sqcup V_K$; intra-core edges E_{intra} describe in-core connectivity and carry weight 1, and inter-core edges E_{inter} connect designated *communication ports* on neighbouring cores and carry a much larger weight $w_{\text{link}} \gg 1$ reflecting the cost of an EPR pair. A communication port is a physical qubit like any other and may also hold a logical (data) qubit between teleportations; the "communication" label only indicates that the port is one end of an inter-core EPR link, not that the slot is reserved. dSABRE treats $G_{\mathcal{A}}$ as a black box and only queries it through three distance tables: physical d_{phys}, intra-core d_{intra}, and core-graph d_{core}. The router therefore extends to *any* DQC topology — arbitrary intra-core graphs, mixed core sizes, sparse inter-core meshes, hierarchical trees of clusters, etc. — by simply supplying these tables.

For experimental simplicity our reference implementation instantiates $G_{\mathcal{A}}$ as a *grid-of-grids* [15]: each core is an $m \times m$ 2D nearest-neighbour grid, and the cores themselves form an $r \times s$ super-grid joined by a small set of inter-core links between specific boundary qubits. Figure 2 shows two concrete topologies used in our experiments.

a) Hierarchical distance precomputation.: Because intra-core and inter-core costs decouple, the all-pairs distance d_{phys} over $G_{\mathcal{A}}$ admits a two-level Dijkstra computation that is substantially cheaper than running Dijkstra on the full $|V|$-vertex graph. First, all-pairs distances are computed once *inside* a single core V_i (size $|V_i| = M$) in $O(M^2 \log M)$ time, giving d_{intra}. Second, the *core graph* (a multigraph of K super-nodes connected by inter-core links) is solved in $O(K^2 \log K)$ time, giving d_{core}. Third, $d_{\text{phys}}(p, q)$ for any $p \in V_i$, $q \in V_j$ is recovered as $d_{\text{intra}}(p, \pi_i^*) + d_{\text{core}}(i, j) + d_{\text{intra}}(\pi_j^*, q)$ minimised over boundary-port pairs (π_i^*, π_j^*) realising the optimal core-graph path. This brings precomputation from $O((KM)^2 \log(KM))$ down to $O(KM^2 \log M + K^2 \log K)$ and is the only architecture-dependent setup dSABRE performs; routing itself uses $O(1)$ table lookups thereafter.

D. Qubit Allocation and Routing

Given circuit DAG C and initial layout $\ell : \mathrm{q}_i \mapsto \mathrm{p}_{\sigma(i)}$ (σ is a permutation on $0, 1, ..., n-1$ with n the number of physical qubits), routing inserts the minimum number of SWAP and teleportation operations so every gate can execute on adjacent physical qubits. Cost model: each local SWAP costs c_{swap}; each inter-core teleport costs c_{tele} (one EPR pair).

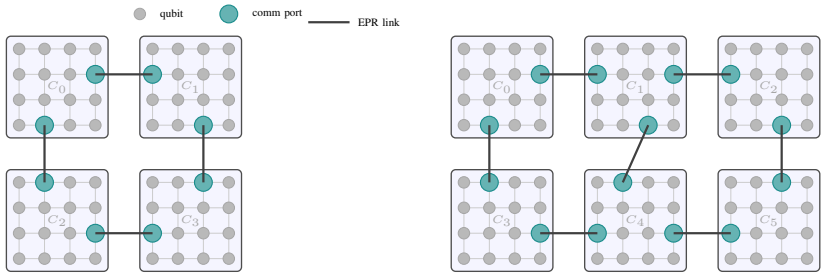
E. SABRE

SABRE [11] is the reference heuristic for single-chip qubit mapping. Qiskit [12] selects it by default for both layout and routing whenever routing is required, and documents it as almost invariably the best performing of its built-in routing plugins; the shipped implementation follows the LightSABRE [13] refinement. Later work has extended rather than displaced it, both on single chips [13] and in the multi-core setting, where the closest prior routers [14], [15] are themselves SABRE derivatives. This is why dSABRE builds on the same scoring form.

LightSABRE [13] optimises SABRE [11] by selecting the SWAP minimising:

$$H = \frac{1}{|F|} \sum_{g \in F} \Delta_F(g) + w \frac{1}{|E|} \sum_{g \in E} \Delta_E(g), \tag{1}$$

where F is the front layer, E the extended lookahead set, and $\Delta(g) = d^g_{\text{new}} - d^g_{\text{old}}$ the change in physical distance



(a) 2×2 B-grid
4 cores, 64 qubits, 4 EPR links

(b) 2×3 H-grid
6 cores, 96 qubits, 7 EPR links

Fig. 2. DQC architectures used in this paper (B-grid and H-grid families introduced by TeleSABRE [15]). Small grey circles are qubits; teal circles are inter-core communication ports; bold lines are EPR-channel links between cores. Each core is a 4×4 grid. (a) 2×2 B-grid. (b) 2×3 H-grid.

between the two qubits of gate g caused by the candidate SWAP (negative when the SWAP moves them closer); d^g is shortest-path distance on the coupling graph. All extended-set gates contribute with equal weight; the $\frac{1}{|E|}$ factor normalises the lookahead term to the same scale as the front term. SABRE also couples its router with an initial-layout optimiser: alternating forward and reverse-circuit passes, each seeded with the previous pass's final layout, anneal towards a layout self-consistent with the circuit's gate structure.

III. DSABRE

dSABRE is a SABRE-style routing algorithm for multi-core processors. Routing assumes a fixed initial layout; we use Qiskit `SabreLayout`, and defer the discussion of layout preparation and the routing pass schedule to Section IV-F.

The core invariants and notation are: (i) the layout $\ell : \mathrm{q}_q \to \mathrm{p}_p$ assigns logical to physical qubits and the inverse ℓ^{-1} records which logical qubit (if any) currently occupies each physical slot; (ii) the working DAG W holds the remaining gates (initially the full circuit) and shrinks as gates are executed; (iii) the front layer F is the set of gates in W with no unexecuted predecessors, and the extended set E is a fixed-size sliding lookahead beyond F. Table I collects every symbol used in this section.

A. Scoring: Potential, Cost, and Capacity

dSABRE moves qubits with two different actions—an intra-core SWAP and an inter-core teleport—and chooses between them by a priority rule rather than a common score: whenever a front-layer gate has both operands in one core, the router applies one intra-core SWAP (Section III-C); only when no such gate remains does it score teleport candidates (Section III-D). The two scorers thus run at different scopes—per-core and taint-restricted for SWAPs, global for teleports—and are never compared against one another.

The intra-core scorer is SABRE restricted to a core, with the depth discount of Eq. 4; it needs no further justification here. The teleport scorer is where the distributed setting bites, and its shape is not a design choice so much as a consequence of three respects in which an inter-core teleport is unlike an intra-core SWAP. *(i) Availability*: a SWAP is legal on every edge of the coupling graph, whereas a teleport can fire only at a communication port, so a candidate must first pay to bring its qubit there. *(ii) Capacity*: on a monolithic chip the layout is a bijection and every qubit has a slot by construction, whereas a core holds M qubits and each teleport consumes one of the destination's free slots, so a candidate must be charged for the capacity it takes. *(iii) Granularity*: a SWAP moves a qubit one edge and the resulting distance change describes the move completely, whereas a teleport delivers the qubit to a specific port that is in general not where its next hop should begin, so the distance change alone under-reports core-level progress. Section III-D adds one term for each, on top of the front-plus-lookahead distance objective inherited from SABRE; the rest of this subsection fixes that objective.

Concretely, this subsection fixes the objective the *teleport* scorer descends, since that is where dSABRE's design choices sit; the SWAP scorer is the standard SABRE rule restricted to a core, with the depth discount of Eq. 4.

Fix a layout ℓ and let $d(g;\ell)$ be the weighted distance between the two operands of a 2Q gate g on the full-chip graph $G_{\mathcal{A}}$ (intra-core edges weight 1, inter-core links weight w_{link}), so $d(g;\ell) = 0$ exactly when g is executable. Define the *routing potential*

$$\Phi(\ell) \;=\; \sum_{g\in F} d(g;\ell) \;+\; w_e \sum_{g\in E} \gamma^{\mathrm{dep}(g)}\, d(g;\ell), \qquad (2)$$

the first term measuring work that must be done now, the second discounted work that is coming, with $\gamma^{\mathrm{dep}(g)}$ discounting a gate by its depth because a deeper gate is more likely to be displaced before it is reached. Φ is a surrogate, not the true cost-to-go: exact at the boundary ($\Phi{=}0$ iff every gate in F is executable), and elsewhere the standard SABRE relaxation, which ignores that qubits contend for the same slots.

A teleport a, together with the staging SWAPs that precede it, transforms ℓ into ℓ_a. Write $\Delta\Phi(a) = \Phi(\ell_a) - \Phi(\ell)$, let $\mathrm{cost}(a)$ be the resource the action consumes, and let $\mathrm{pen}(a)$ charge the capacity constraint that Φ omits. dSABRE greedily minimises

$$s(a) \;=\; \underbrace{\mathrm{cost}(a)}_{\text{resource spent}} \;+\; \underbrace{\mathrm{pen}(a)}_{\text{capacity}} \;+\; \underbrace{\Delta\Phi(a)}_{\text{progress bought}} . \qquad (3)$$

The three groups answer three questions—what the move costs, whether it is legal in practice, and how much closer it gets us. Section III-D instantiates each: $\mathrm{cost}(a)$ is the

staging count d_{prep}, $\text{pen}(a)$ is the capacity penalty c_{cap}, and $\Delta\Phi(a) = \Delta_F + w_e\tilde{\Delta}_E$ is the induced change in the two sums of Eq. 2, restricted to the gates on the teleported qubit, which are the only gates the move affects. The sums are left un-normalised: the candidate set spans cores, so no per-core cardinality is shared across candidates, and the size of E is instead bounded by the cap $|E|_{\max}$. Within one iteration every candidate spends exactly one EPR pair, so the teleport's own EPR cost is constant across the arg min and does not appear in Eq. 3; $\text{cost}(a)$ carries only the staging SWAPs, which do vary between candidates.

Two properties of this score are what the experiments later test. First, the capacity term is not a tie-breaker bolted onto a distance heuristic: it is the one place where the architecture's finiteness enters an otherwise pure relaxation, and it is *graded* in the landing core's free-slot count rather than switching on at a fixed occupancy. Section IV-E finds it the largest single contributor to dSABRE's EPR advantage, ahead of the lookahead term and well ahead of every other component. Second, the lookahead enters only through $\gamma^{\text{dep}(g)}$, so a gate's influence decays with its DAG depth; Section III-E shows that this discount is well-defined only if E is built in BFS layers.

The mechanisms of Sections III-F and III-G exist because a greedy minimiser of Eq. 3 can reach states where Φ stops being a usable surrogate—every cheap action blocked by saturated cores, or no action reducing $|W|$ at all. They enter differently: congestion relief contributes extra candidates, scored by Eq. 6 plus a congestion bonus and competing in the same arg min (Section III-F), whereas checkpoint–rollback acts outside the score entirely (Section III-G). We report the value of both in terms of feasibility and completion rather than as EPR reductions.

At the intra-core level dSABRE's SWAP-selection objective inherits the front-layer / extended-set scoring of Eq. 1, with two refinements: a decay-weighted lookahead ($\gamma^{\text{dep}(g)}$ down-weights deep successors) and taint propagation that excludes qubits already entangled with cross-core gates (Section III-C). Inter-core movement is handled by a separate teleportation scorer (Section III-D).

B. Workflow

Algorithm 1 and Figure 3 sketch the main routing loop. Each iteration runs four phases: **(P1)** drain F (execute 1Q gates and any 2Q gate whose operands are already adjacent in the same core, iterated to a fixed point); **(P2)** partition the remaining front into F_{intra} (both operands in one core) and F_{inter} (operands in different cores); **(P3)** if $F_{\text{intra}} \neq \emptyset$ apply one intra-core SWAP (Section III-C), else score teleportation candidates over F_{inter} together with proactive relief candidates and apply the cheapest (Section III-D); **(P4)** checkpoint (ℓ, W) if $|W|$ decreased, otherwise trigger checkpoint–rollback after L_{deadlock} stalled iterations and abort after $N^{\max}_{\text{backup}}$ failed recoveries. The two scoring functions operate at different scopes: the intra-core scorer is per-core, restricted to intra-core continuations (qubits involved in inter-core gates are tainted), and front-/extended-set averaged in the SABRE style; the inter-core scorer evaluates moves across cores using the global F_{inter} and a global BFS-layer extended set (Section III-E), with $\tilde{\Delta}_E$ entering as an un-averaged decay-weighted sum because the cross-core lookahead window is small and its size already enters implicitly via the $|E|_{\max}$ cap.

TABLE I
NOTATION FOR SECTION III, WITH THE DEFAULT VALUE OF EACH HYPERPARAMETER. DEFAULTS ARE USED UNCHANGED IN EVERY EXPERIMENT AND ON EVERY ARCHITECTURE.

Symbol	Meaning	Default
Routing state		
$\ell,\ \ell^{-1}$	layout logical→physical, and its inverse	—
W	working DAG of unexecuted gates	—
$F,\ E$	front layer; extended (lookahead) set	—
$F_c,\ E_c$	their restrictions to core c	—
$K,\ M$	number of cores; qubits per core	—
Distances and scores		
$G_{\mathcal{A}}$	full-chip graph (Section II-C)	—
$d(g;\ell)$	weighted distance between operands of g	—
$d_{\text{intra}},\ d_{\text{core}}$	intra-core and core-graph distance	—
$\text{dep}(g)$	depth of g (BFS layer, inter-core set)	—
Φ	routing potential, Eq. 2	—
Δ_F	change in d over the front gate	—
$\tilde{\Delta}_E$	decay-weighted change over E, Eq. 4	—
d_{prep}	staging + eviction SWAPs before a teleport	—
c_{cap}	capacity penalty on the landing core	—
g_{hop}	directional hop-gain reward	—
f_{dst}	free slots in the landing core c_{next}	—
$\pi_s,\ \pi_d$	source/destination communication ports	—
n_s	staging slot: neighbour of π_s nearest p_1	—
Hyperparameters		
c_{swap}	SWAP cost	3
c_{tele}	teleport (EPR) cost	10
τ	capacity threshold	3
c_{pen}	capacity penalty multiplier	15
w_h	hop-gain weight	5
w_{link}	inter-core link weight	10
w_e	extended-set weight	0.25
L	extended-set capacity	20
γ	lookahead decay	0.9
L_{deadlock}	stalled iterations before rollback	50
$N^{\max}_{\text{backup}}$	maximum rollbacks	50

C. Intra-Core SWAP Heuristic

When the front contains a gate whose operands share a core but are not adjacent, dSABRE chooses an intra-core SWAP using the standard SABRE heuristic, restricted to the relevant core. Let $F_c = \{g \in F_{\text{intra}} : \text{core}(g) = c\}$ be the front gates in core c and $E_c = (g_0, g_1, \ldots)$ the corresponding intra-core extended set, an *ordered list* of at most L gates (default L=20) collected in topological order. The traversal stops including a gate the moment either of its qubits has previously appeared in a cross-core gate in the working DAG (taint propagation), ensuring E_c contains only gates that remain local to core c. Each gate g_i is assigned a depth $\text{dep}(g_i)$, the number of intra-core gates on the longest qubit-wire path from the front layer to g_i (front layer = depth 0; first successor = depth 1). The lookahead contribution is

$$\tilde{\Delta}^{c}_{E} = \sum_{i=0}^{|E_c|-1} \gamma^{\text{dep}(g_i)} (d^{g_i}_{\text{new}} - d^{g_i}_{\text{old}}), \tag{4}$$

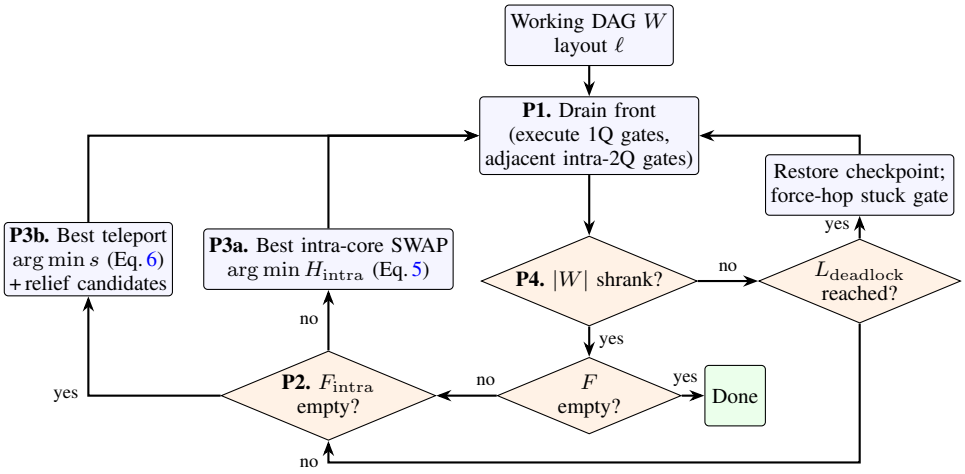


Fig. 3. The dSABRE routing workflow. Each iteration drains the front layer (P1), classifies the remaining front into intra-core and inter-core gates (P2), applies one SWAP or one teleport (P3), and checkpoints when forward progress is made (P4).

Algorithm 1 dSABRE `route`(C, ℓ_0)

$W \leftarrow$ working copy of circuit DAG C; $\ell \leftarrow \ell_0$
$C_{\text{out}} \leftarrow$ empty routed circuit on the physical qubits
save checkpoint (ℓ, W)
while W has unexecuted gates **do**
  drain F (execute 1Q gates and adjacent intra-core 2Q gates; append them to C_{out}) *(P1)*
  if W empty **then**
    break
  end if
  compute F, partition into $F_{\text{intra}}, F_{\text{inter}}$ *(P2)*
  if $F_{\text{intra}} \neq \emptyset$ **then**
    compute per-core intra-only extended set E_c for each active core c *(P3a)*
    pick best SWAP (u, v) within a core (Eq. 5)
    apply SWAP(u, v) to ℓ; append SWAP(u, v) to C_{out}
  else if $F_{\text{inter}} \neq \emptyset$ **then**
    compute global E if not cached *(P3b)*
    generate candidates over F_{inter} and relief moves
    pick lowest-score candidate (Eq. 6)
    apply teleport to ℓ; append teleport (staging SWAPs + teledata) to C_{out}
  end if
  if $|W|$ decreased **then**
    save checkpoint *(P4)*
  else if L_{deadlock}=50 stalled iterations **then**
    restore checkpoint; `backup_plan`$(W, \ell, C_{\text{out}})$ // *abort after* $N^{\max}_{\text{backup}}$=50
  end if
end while
return $(\ell, C_{\text{out}}, \text{metrics})$

where $\gamma \in (0, 1]$ is the lookahead decay and the distance terms use the intra-core graph. The tilde marks the departure from SABRE's flat extended-set weighting: rather than treating all lookahead gates equally, the exponential factor $\gamma^{\text{dep}(g_i)}$ down-weights gates far from the front so that near-term successors dominate the lookahead signal — a deeper gate is more likely to be displaced by intervening routing decisions and so contributes less reliable information about the right SWAP now. The same weighted scheme is reused by the inter-core scorer (Section III-D); Section III-E explains how $\gamma^{\text{dep}(g)}$ interacts with the BFS-layer extended set, where $\text{dep}(g)$ is the BFS depth rather than an iteration index. For every neighbour pair (u, v) within a core, the score

$$H_{\text{intra}} = \frac{\Delta^{\text{c}}_F}{|F_{\text{c}}|} + w_e \frac{\tilde{\Delta}^{\text{c}}_E}{|E_{\text{c}}|} \tag{5}$$

is the SABRE objective for intra-core scoring with the lookahead-decay weighting of Eq. 4. The lowest-score SWAP is applied; tie-breaking is by enumeration order. Because the scorer is restricted to a single core and the candidate set is the $O(M)$ intra-core neighbour pairs, the per-core selection is trivially parallel across active cores.

D. Inter-Core Teleportation Scoring

For each gate $g \in F_{\text{inter}}$ on logical qubits q_1, q_2, let c_{src} and c_{tgt} be the cores currently hosting q_1 and q_2 respectively (i.e. $\ell(q_1) \in c_{\text{src}}$ and $\ell(q_2) \in c_{\text{tgt}}$, where ℓ is the current layout). Both endpoints are tried as teleportation sources. For the qubit chosen as source (say q_1 at physical position $p_1 = \ell(q_1)$), the router enumerates every neighbouring core c_{next} and every inter-core link (π_s, π_d) between c_{src} and c_{next}. Each such triple $(q_1, c_{\text{next}}, (\pi_s, \pi_d))$ is one *candidate*; Figure 4 illustrates the structure. Executing a candidate involves two steps: (1) intra-core SWAPs that move q_1 from p_1 to the *staging slot* n_s, the neighbour of π_s closest to p_1; and (2) a teleportation that consumes the EPR pair on link (π_s, π_d) and delivers q_1 from n_s to π_d in c_{next}. Each candidate is scored as:

$$s = \underbrace{d_{\text{prep}}}_{\text{staging}} + \underbrace{c_{\text{cap}}}_{\text{capacity}} \underbrace{- g_{\text{hop}}}_{\text{direction}} + \underbrace{\Delta_F + w_e \tilde{\Delta}_E}_{\Delta\Phi\ \text{(front + lookahead)}}, \tag{6}$$

where lower scores are preferred. The last group is the front-plus-lookahead distance objective inherited from SABRE (Eq. 1), and it is the whole score on a monolithic chip. The

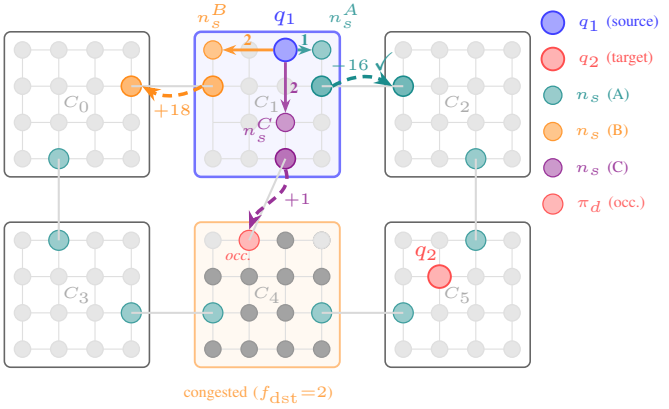


Fig. 4. How dSABRE chooses where to teleport a qubit. A single cross-core gate $g{=}(q_1, q_2)$ is pending, with q_1 in core C_1 (blue border) and its partner q_2 in C_5; the router scores the three links leaving C_1 and takes the cheapest by Eq. 6. Executing any one of them takes two steps: intra-core SWAPs first move q_1 to the staging slot n_s (count on the solid arrow), then a teleport along the link (π_s, π_d) carries it into the next core (dashed arc, annotated with the resulting score s). The three candidates isolate one decision factor each, and correspond row by row to Table II. **A** (teal) moves towards C_5 and wins, $s_A{=}{-}16$. **B** (orange) moves to C_0, away from C_5, so its front-layer term turns positive. **C** (violet) makes the same progress as A but lands in the nearly-full core C_4 (orange border, two free slots), where the capacity penalty outweighs the progress. The red port marked *occ.* is already occupied, so C also pays an extra eviction SWAP.

first three are the three differences of Section III-A, in order: d_{prep} for availability, c_{cap} for capacity, g_{hop} for granularity. Equivalently they are the three groups of Eq. 3—what the move spends, whether it is legal-in-practice, and what it buys. Each term is defined as follows.

Staging cost d_{prep} is the number of intra-core SWAPs needed to move q_1 from p_1 to staging slot n_s, plus eviction SWAPs to free the communication ports if occupied:

$$d_{\text{prep}} = d_{\text{intra}}(p_1,\, n_s) + d_{\text{evict}}(\pi_s) + d_{\text{evict}}(\pi_d).$$

Capacity penalty $c_{\text{cap}} = c_{\text{pen}}\ \max(0,\, \tau - f_{\text{dst}})$ discourages routing into nearly-full cores. f_{dst} is the number of free slots in the immediate landing core c_{next} (not the final partner-qubit core c_{tgt}), τ the capacity threshold, and c_{pen} a cost multiplier. This prevents progressive crowding of hot cores that leads to deadlock.

Front-layer change $\Delta_F = d^g_{\text{new}} - d^g_{\text{old}}$ is the change in weighted physical distance between q_1 and q_2 after the teleport (negative when they are brought closer), computed on the full-chip graph $G_{\mathcal{A}}$ (intra-core edges weight 1, inter-core links weight w_{link}). There is exactly one front-layer gate involving the moving qubit q_1: because any two gates that share a qubit are ordered by a dependency edge in the DAG, at most one of them can be unblocked (i.e. in F) at any time, and $g = (q_1, q_2) \in F_{\text{inter}}$ is already that gate by the premise of this scoring step.

Hop gain $g_{\text{hop}} = w_h\,(d_{\text{core}}(c_{\text{src}}, c_{\text{tgt}}) - d_{\text{core}}(c_{\text{next}}, c_{\text{tgt}}))$ is a position-independent directional reward. It is the term for the granularity difference, and it exists because $\Delta\Phi$ is evaluated at a *specific* landing port π_d: when π_d is far from the best staging position for the next hop, Δ_F under-reports the core-level progress the move actually buys. Whether that correction is worth a hyperparameter is an empirical question, and we tested it directly by re-running every suite with $w_h{=}0$. On both B-grids the term is *exactly* inert—all twelve circuits tie to the EPR—because with $w_{\text{link}}{=}10$ the weighted distance in Δ_F already resolves core-level direction on a four-core chip. Its value appears only as cores and paths grow: on the 64-qubit H-grid, removing it costs $+2.2\%$ on the geometric mean, concentrated in the two largest circuits ($+18.6\%$ on the 13 040-CX Multiplier, $+6.4\%$ on Random) and partly offset elsewhere (-4.1% on AE). We therefore keep the term, and read this as a scale effect rather than a tuning artifact: the larger the cores, the further a landing port can sit from the next staging position, and the more Δ_F alone understates a good move. Section IV-E reports the ablation.

Decay-weighted lookahead. $\tilde{\Delta}_E$ is the same decay-weighted lookahead introduced in Section III-C (Eq. 4), reused here with two adaptations for the inter-core scorer: E is now the inter-core extended set (Section III-E) rather than the per-core taint-propagated list, and the sum is restricted to gates involving the teleported qubit, since only those gates are affected by the candidate move. The same γ controls how rapidly deeper successors are discounted.

a) Running example.: Figure 4 illustrates Eq. 6 on the 2×3 H-grid with defaults $w_h{=}5$, $w_{\text{link}}{=}10$, $\tau{=}3$, $c_{\text{pen}}{=}15$. Gate $g{=}(q_1, q_2)$ has $d_{\text{core}}(C_1, C_5){=}2$ and is the only pending gate on q_1, so Δ_F is evaluated from g alone and $\tilde{\Delta}_E{=}0$ (empty extended set). With $p_1{=}(0,2) \in C_1$ and $p_2{=}(1,1) \in C_5$, the shortest path on $G_{\mathcal{A}}$ gives $d^g_{\text{old}}{=}28$ (via $C_1{\to}C_2{\to}C_5$). Table II scores the three outgoing-link candidates, and shows the three groups of Eq. 3 deciding between them: A wins by combining the cheapest staging with the largest progress and a positive hop gain; B loses because q_1 moves away from C_5, so both Δ_F and g_{hop} turn against it; C buys the same progress as A but lands in a nearly-full core, and the capacity term alone reverses the decision.

TABLE II
SCORE BREAKDOWN FOR THE THREE CANDIDATES OF THE RUNNING EXAMPLE (FIGURE 4). LOWER IS BETTER; CANDIDATE A IS SELECTED.

Candidate	next core	direction	d_{prep}	c_{cap}	g_{hop}	Δ_F	score s
A	C_2	towards C_5	1	0	+5	−12	**−16**
B	C_0	away from C_5	2	0	−5	+11	18
C	C_4	towards C_5	3	15	+5	−12	1

E. Inter-Core Extended-Set Construction

The lookahead $\tilde{\Delta}_E$ in Eq. 6 depends on how the inter-core extended set E is built. SABRE [11] fills E in topological order, interleaving all wires uniformly; TeleSABRE [15] uses BFS layers but within each layer emits gates in arbitrary order and weights each by its iteration index g via $(1 + g/10)$ rather than by depth. Both schemes can deprioritise the very follow-on gates a candidate teleport would help, causing $\tilde{\Delta}_E$ to undercount its benefit.

dSABRE keeps a BFS-layer expansion of the remaining DAG with two refinements: (i) within each layer, gates sharing a qubit with the front are emitted first; (ii) every gate in BFS

layer k is assigned $\mathrm{dep}(g)=k$, used in the decay $\gamma^{\mathrm{dep}(g)}$, so the size-L truncation cuts cleanly at a layer boundary and weighting decays exponentially with depth rather than growing linearly with iteration index. It adds no hyperparameter and runs in $O(|E|_{\max})$ time per call when remaining in-degrees are maintained incrementally: BFS peels zero-in-degree gates layer by layer, each emitted gate triggering $O(1)$ decrements on its successors. The empirical contribution over the topological variant is reported in Section IV-E.

The reason the construction matters is structural rather than incidental. In Eq. 2 a gate's influence is set by $\gamma^{\mathrm{dep}(g)}$, so the lookahead is only meaningful if the gates that survive the size-L truncation are the shallow ones. BFS layering guarantees exactly that; topological order does not.

Proposition 1 (Depth monotonicity): Let E_{BFS} and E_{topo} be the size-L extended sets built by BFS-layer expansion and by topological order. Then E_{BFS} contains every gate of depth $\leq k$ before any gate of depth $> k$, for every k. No topological filling has this property: for every L there is a DAG in which E_{topo} omits a depth-1 gate while admitting L gates of depth up to L.

The proof is in the online appendices.

This is why the gain from BFS layering grows with circuit size (Section IV-E): the deeper and wider the remaining DAG, the further a topological sweep drifts from the active front before the budget L is spent. The proposition bounds the ordering, not the EPR count—routing quality still depends on the circuit—but it identifies the mechanism the ablation measures.

F. Proactive Congestion Relief

In dense layouts, a core can become saturated: many pending gates demand qubits from it simultaneously and the core has no free slots to absorb incoming teleports. Standard reactive scoring (Eq. 6) does not address this until the bottleneck triggers a deadlock.

dSABRE adds a *proactive congestion relief* mechanism that fires alongside the gate-driven scoring of Eq. 6.

Demand vector. At each iteration the router computes a demand vector $d[c]$ over all cores by counting inter-core gates in the front and lookahead windows whose shortest core-graph path traverses c. $d[c]$ is therefore an upper bound on the number of upcoming teleports that will need to pass through c, regardless of whether c itself hosts an operand.

Trigger. A core c is flagged when both $d[c] \geq \theta_d$ and $f_c \leq \theta_f$ hold (defaults $\theta_d{=}\theta_f{=}1$, used unchanged in every experiment): incoming demand combined with low free capacity. A flagged core is relieved only into a neighbour holding at least $\theta_r{=}3$ free slots, so relief never creates the congestion it is meant to remove. The reactive scorer alone would only respond once f_c hits zero and the next gate-driven teleport fails to find a landing port; firing earlier on the conjunction avoids the eviction-cascade pattern that triggers deadlock.

Relief candidates. For each flagged core, the router generates teleports that move the most-idle logical qubits in c — those whose next pending gate has the largest DAG depth from the front — into a less-loaded neighbour. Each candidate is scored by Eq. 6 with $\Delta_F{=}0$ — victims are restricted to qubits absent from the front layer, so no pending gate motivates the move and the hop-gain term is omitted too, there being no gate-defined target core. The decay-weighted lookahead $\tilde{\Delta}_E$ is retained, so that when a victim does have an upcoming gate within the extended-set window the landing port is preferred to be closer to its next partner. Three bonuses are then subtracted: a *relief bonus* $b_r\,(d[c] - f_c + 1)$ proportional to the demand–capacity imbalance ($b_r{=}8$); a *next-use* term b_{dep} times the victim's DAG depth to its next pending gate, preferring qubits not needed soon ($b_{\mathrm{dep}}{=}0.5$); and a *gradient* term b_g times the busyness difference between the congested and receiving cores, where busyness is occupancy plus demand, preferring the emptiest available neighbour ($b_g{=}1$). Relief candidates compete directly against gate-driven candidates and win only when these bonuses outweigh the staging cost; the mechanism therefore introduces no extra teleports when the front-layer scorer already has a cheap on-target move available. On the per-core adaptive layout the ablation in Section IV-E shows relief is close to gmean-neutral (disabling it *lowers* 64q gmean EPR by 8.7%): the layout now gives every core escape capacity, so the standing-congestion the relief mechanism was designed to pre-empt is largely absent, and its remaining contribution is confined to individual hard instances such as the 64q Random circuit.

G. Checkpoint–Rollback Deadlock Escape

When Eq. 6 stalls, dSABRE falls back to LightSABRE's release-valve idea [13]: force shortest-path progress on the most-stuck front-layer gate. We wrap it in a checkpoint: every iteration that reduces $|W|$ saves (ℓ, W), and after L_{deadlock} stalled iterations the router restores the checkpoint before forcing intra-core SWAPs or hop-by-hop teleportation along $\mathrm{core\_path}(c_1, c_2)$, aborting after $N^{\max}_{\mathrm{backup}}$ failed recoveries. The escape is dormant on best-EPR runs and only salvages individual SabreLayout seeds on dense large circuits (e.g. one of three seeds on 200q QFT, with 17 activations).

Its purpose is not to lower EPR count but to make the greedy rule of Eq. 3 safe to deploy. A pure greedy minimiser has no progress guarantee: nothing in Eq. 3 prevents a cycle of moves that each look locally attractive while $|W|$ never shrinks. The escape bounds what such a stall can cost.

Proposition 2 (Forced progress): Let D be the diameter of the core graph, and consider one invocation of the escape on a stuck front gate $g = (q_1, q_2)$ whose operands lie in cores $c_1 \neq c_2$. Suppose each core along $\mathrm{core\_path}(c_1, c_2)$ either has a free slot or has a neighbouring core with one *(capacity condition)*. Then the escape brings both operands into a common core using at most $2D$ EPR pairs, after which g becomes executable within a number of intra-core SWAPs bounded by the core diameter, so $|W|$ strictly decreases. Independently of the capacity condition, dSABRE always halts.

The proof is in the online appendices.

The distinction in the last sentence is deliberate. Termination is unconditional, but *completion* rests on the capacity condition, which can fail if a region of the chip becomes globally saturated—the evacuation step searches only the neighbours

of a full core, not the whole chip. This is a second reason the reserved escape slots of Section IV-F matter: they keep the condition satisfied in practice. Empirically the budget is never binding: no run in this paper reached $N_{\text{backup}}^{\max}=50$, and the escape is dormant on best-EPR runs. TeleSABRE offers a fixed safety-valve iteration cap in place of any progress argument, consistent with its failures to converge on the denser circuits of Section IV-B.

H. Computational Complexity

Let $N=|C|$ be the gate count, n the circuit width, K the number of cores and M qubits per core ($P=KM$). Architecture distance tables are precomputed once in $O(KM^2\log M + K^2\log K)$ (Section II-C) and are not charged per circuit. Per iteration, inter-core scoring generates $O(n)$ candidates and intra-core scoring $O(M)$ per core, giving $O(Mn+P)$, which is $O(Mn)$ whenever $n \geq K$ as on all our suites. With physical diameter $O(\sqrt{P})$ bounding the inserted operations,

$$T_{\text{total}} = O\Big(N\sqrt{P}\cdot Mn\Big). \tag{7}$$

Restricting intra-core scoring to core-local edges yields a factor-K per-iteration improvement over flat SABRE [11] on the same P qubits, growing with core count. The derivation is in the online appendices.

IV. Experimental Evaluation

A. Setup

Benchmarks. We use three circuit sets from the MQT Bench suite [17], all in the IBM native-gate set (Qiskit opt3 transpilation, measurements and barriers removed).

Suite 1 (25-qubit): quantum amplitude estimation (AE), quantum Fourier transform (QFT), quantum neural network (QNN), random circuit (Random), GHZ state preparation, and graph-state preparation.

Suite 2 (36-qubit): six circuits spanning a broader range of algorithmic families and CX densities: Bernstein–Vazirani (BV, linear chain), Deutsch–Jozsa (DJ, oracle), W-state preparation (cascaded star), VQE with SU(2) ansatz (layered variational), QAOA (dense all-to-all), and quantum phase estimation (QPEexact, hierarchical).

Suite 3 (64-qubit): the same six circuit families as Suite 1 scaled to 64 logical qubits (AE, QFT, QNN, Random, GHZ, Graphstate), run on the larger H-grid architecture, extended with three further circuits that broaden the evaluation towards application-oriented workloads and complete the algorithmic-class coverage at the largest scale: QPEexact (phase-estimation family, the class on which gate teleportation is strongest), QAOA (dense max-cut cost layers with shared-control ZZ structure), and a 16-bit Multiplier (arithmetic, the suite's largest circuit at 13 040 CX). QPEexact is the MQT-Bench v1.1.0 nativegates/opt3 circuit; QAOA and Multiplier are generated with MQT-Bench v2.2.2 in the same format (basis $\{R_z, SX, X, CX\}$, Qiskit optimisation level 3).

Per-circuit properties (qubits, CX count, depth, CX per qubit) are tabulated in the online appendices.

Architecture. Suites 1 and 2 use a 2×2 B-grid: four 4×4 cores (64 physical qubits, 4 EPR links). Suite 3 uses a 2×3 H-grid: six 4×4 cores (96 physical qubits, 7 EPR links). See Figure 2 for the full qubit-level topology. The 25q suite on the B-grid and the 64q suite on the H-grid follow the benchmark setup of TeleSABRE [15] (we add a 36q B-grid suite to broaden circuit diversity).

Baselines. We compare against two state-of-the-art distributed routers: (i) TeleSABRE [15], a SABRE-style router that jointly optimises intra-core SWAPs, teledata (qubit moves), and telegate (remote CX execution) operations; and (ii) `pytket-dqc` [4], a Quantinuum library that distributes circuits via *gate teleportation* with multi-gate amortisation using KaHyPar hypergraph partitioning. TeleSABRE is run using its compiled C binary with the `optimize_initial` flag enabled (built-in layout heuristic + 3-success sampling) on the same device. For `pytket-dqc` we report its *strongest* distributor, CoverEmbeddingSteinerDetached (Steiner-tree gate embedding with detached gates), best of 5 random seeds, falling back to the cheaper PartitioningHeterogeneous only where the embedding method exceeds a 20-minute budget or cannot embed the circuit; such cells are marked in the tables. This is a harder baseline than the partitioning allocator used in the submitted version, and it wins several structured circuits; we report it because a comparison against a deliberately weakened configuration would not answer the question the tables are asked to answer. Its hypergraph encoding requires a minor weight-0 patch to run on the KaHyPar $\geq$ 1.3.5 release we use.

What one EPR pair buys. All routers report in the same unit (one EPR pair = one e-bit; "EPR" and "e-bit" are used interchangeably), but what each unit *buys* differs, and no cross-compiler EPR comparison is meaningful without stating the difference. Table III summarises the four cost models in play. dSABRE uses *teledata*: one EPR per inter-core hop relocates a logical qubit, after which subsequent intra-core gates on it are free. `pytket-dqc` uses *gate teleportation*: one EPR distributed along a Steiner tree of cores executes a whole hyperedge of CNOTs sharing one control, paying one EPR per Steiner edge rather than per CNOT. TeleSABRE uses both primitives and we count each operation once. Circuits with isolated inter-core gates favour teledata; circuits with long shared-control CNOT runs (QFT, QPE) favour gate teleportation.

Four differences shape how the `pytket-dqc` column should be read, and together they are the reason we do not headline it. (i) *Work per unit differs*: one e-bit buys a whole hyperedge of gates, one teledata EPR buys one relocation, so the columns are unit-consistent but not work-consistent. (ii) *Scope differs*: `pytket-dqc` emits an e-bit count only and does not model intra-core SWAP routing at all, so its column omits a cost dSABRE pays and reports. (iii) *The architecture model differs*: its `NISQNetwork` records *which* core pairs are connected but not *how many* physical links join them, and its per-server communication capacity defaults to *unbounded*. (iv) The asymmetry is also *temporal*: a gate-teleportation e-bit must stay entangled for the full span of the hyperedge it serves, whereas a teledata EPR is consumed instantaneously, so `pytket-dqc`'s counts presuppose an entanglement lifetime the hardware may not provide. Section IV-C confronts (iii) by rebuilding the network to dSABRE's physical constraints, and

TABLE III
WHAT ONE EPR PAIR BUYS UNDER EACH COMPILER'S COST MODEL. COUNTS ARE UNIT-CONSISTENT ACROSS ROWS BUT PURCHASE DIFFERENT AMOUNTS OF WORK, AND ONLY THE FIRST TWO ROWS CHARGE INTRA-CORE SWAPS.

Compiler	Primitive	One EPR pair buys	SWAPs
dSABRE	teledata	one qubit relocation, one hop	counted
TeleSABRE	teledata + telegate	a relocation, or one remote CX	counted
`pytket-dqc`	gate teleport	one Steiner edge of a hyperedge	not modelled
DMapS	remote SWAP	half a two-qubit exchange (2 EPR)	counted

Section IV-B addresses (i)–(ii) with a layout-controlled comparison and (iv) with a lifetime sweep. Against TeleSABRE, which shares dSABRE's cost model, physical architecture and intra-core SWAP accounting, none of these corrections is needed and the comparison is direct; that is why it is the one we headline.

Parameters. Table I lists all hyperparameters; the defaults are used unchanged on every suite and architecture. dSABRE is deterministic given a fixed layout; we run it with three SabreLayout seeds and report the best EPR count (lower = better). TeleSABRE is stochastic and is run with three random seeds; again the best result is reported, following established practice [11]. Layout passes: 2 (forward–backward–forward).

On best-of-N reporting. Each router uses its authors' convention: best-of-3 seeds for dSABRE and TeleSABRE [15], best-of-5 for `pytket-dqc` [4]. This reflects the operational cost rather than per-attempt variance. The advantage over TeleSABRE is robust to this choice: replacing best with median on the original five-circuit TeleSABRE-matched 64q set shifts the gmean gap from -52.6% to -50.2% vs. TeleSABRE, still a large reduction. The comparison against `pytket-dqc` is method-dependent (Tables IV–IV) and is discussed per suite above.

B. Main Results

TeleSABRE [15] supports three operation types: intra-core SWAPs, teledata (qubit movement, 1 EPR), and telegate (remote CX execution via the cat-entangler protocol, 1 EPR). We report TeleSABRE's EPR count as the sum of teledata and telegate operations, so each EPR pair is counted exactly once. Unless stated otherwise, dSABRE is run with its default initial-mapping configuration: Qiskit `SabreLayout` on the corners-removed architecture graph, best of three seeds, followed by SABRE-style forward→backward→forward routing passes (Section IV-F). Subsequent tables and ablations inherit this default. We report two metrics in the main tables: EPR pairs and intra-core SWAPs. Cost analysis is deferred to Section IV-I.

25q. dSABRE reduces geometric-mean EPRs over TeleSABRE by **40.6%** and over pytket-dqc by **44.3%** (Table IV). Per-circuit reductions versus TeleSABRE range from parity on AE to -81.8% on Graphstate, with no regressions on this suite. Against pytket-dqc, given the device's own capacity, dSABRE wins on five of six circuits and ties on Graphstate: state teleportation amortises a single EPR pair over many subsequent local gates, and at this scale that outweighs the gate-embedding savings pytket-dqc extracts by reusing shared control qubits. Under the permissive default the margin would read -56.6%; we report the tighter figure.

TABLE IV
MAIN RESULTS. EPR = TELEDATA + TELEGATE FOR TELESABRE, TELEDATA FOR DSABRE, E-BITS FOR `PYTKET-DQC`; SWAP COUNTS INTRA-CORE SWAPS ONLY (`PYTKET-DQC` DOES NOT MODEL INTRA-CORE ROUTING, TABLE III). NEGATIVE ΔEPR FAVOURS DSABRE. TELESABRE IS BEST-OF-3 SEEDS, DSABRE BEST-OF-3 SABRELAYOUT SEEDS UNDER THE FWD→BWD→FWD SCHEDULE. `PYTKET-DQC` IS BEST-OF-5 WITH ITS STRONGEST COMPLETED DISTRIBUTOR, RUN ON THE NETWORK MODEL THAT MATCHES THIS DEVICE—PER-CORE DATA CAPACITY $M - \deg(c)$ AFTER RESERVING COMMUNICATION PORTS, COMMUNICATION CAPACITY $\deg(c)$ (SECTION IV-C); THIS IS A STRONGER BASELINE THAN THE PERMISSIVE DEFAULT AND THE FIGURES BELOW ARE CORRESPONDINGLY TIGHTER THAN THOSE IN THE CONFERENCE VERSION. *TELESABRE FAILS TO CONVERGE; ITS GEOMETRIC MEAN IS TAKEN OVER THE CIRCUITS IT COMPLETES, SO A SECOND ROW REPORTS THE `PYTKET-DQC` COMPARISON OVER ALL CIRCUITS DSABRE ROUTED. ‡COVEREMBEDDINGSTEINERDETACHED DID NOT COMPLETE WITHIN ITS BUDGET; PARTITIONINGHETEROGENEOUS IS REPORTED. §ARCHITECTURE NOT COVERED BY THE PHYSICAL-MODEL SWEEP; THE PERMISSIVE-DEFAULT FIGURE IS SHOWN.

		TeleSABRE		dSABRE		`pytket-dqc`	ΔEPR (%)	
Circuit	CX	EPR	SWAP	EPR	SWAP	e-bits	vs. TS	vs. tket
25 logical qubits, B-grid 2×2 of 4×4 cores (64 physical)								
ae	558	23	297	23	284	48	+0.0	−52.1
ghz	24	2	29	1	6	2	−50.0	−50.0
graphstate	25	11	51	2	15	2	−81.8	+0.0
qft	580	39	355	33	334	57	−15.4	−42.1
qnn	1223	51	587	48	663	127	−5.9	−62.2
random	1124	292	1273	178	1241	314	−39.0	−43.3
gmean (6)		25.8	221	15.3	138	27.5	**−40.6**	**−44.3**
36 logical qubits, same B-grid								
bv	17	5	34	1	12	1	−80.0	+0.0
dj	35	3	53	3	23	2	+0.0	+50.0
qaoa	1200	232	1027	137	1015	144	−40.9	−4.9
qpeexact	1019	100	771	65	586	52	−35.0	+25.0
vqe_su2	105	16	80	10	56	12	−37.5	−16.7
wstate	70	12	88	6	25	8	−50.0	−25.0
gmean (6)		20.1	147	10.8	78	10.6	**−46.3**	**+1.8**
64 logical qubits, H-grid 2×3 of 4×4 cores (96 physical)								
ae	1962	323	1508	242	1493	142	−25.1	+70.4
ghz	63	11	111	9	57	9	−18.2	+0.0
graphstate	64	76	379	16	89	7	−78.9	+128.6
qft	1966	410	1872	204	1563	204	−50.2	+0.0
qnn	8126	1365	6292	508	5229	605‡	−62.8	−16.0
random*	1627	—	—	721	3608	925‡	—	−22.1
qpeexact	2139	310	1624	271	1792	118	−12.6	+129.7
qaoa*	3920	—	—	529	3688	473‡	—	+11.8
multiplier	13040	2202	11232	1514	10109	2891‡	−31.2	−47.6
gmean (7)		269.5	1452	147.0	1016	120.9	**−45.5**	**+21.6**
gmean (9)		—	—	202.2	1350	176.3	—	**+14.7**
Large QFT (banded, range 19), H-grid, each router from its own layout								
100q	3420	248	2414	223	2723	137	−10.1	+62.8
200q*	7220	—	—	834	7043	456‡	—	+82.9
360q	13300	1109	15098	626	13433	285‡	−43.6	+119.6

36q. dSABRE reduces geometric-mean EPRs by **46.3%** vs. TeleSABRE, and is at parity with pytket-dqc ($+1.8\%$) once that compiler is given the device's own capacity (Table IV); under the permissive default the same comparison reads -12.3%. Per-circuit reductions versus TeleSABRE are large (-35% to -80%, DJ at parity). The simple structured circuits BV, VQE-SU2, and W-state are essentially solved (1 EPR on BV; 6–10 EPRs on the others). Against pytket-dqc's Steiner-embedding distributor the picture is mixed: dSABRE matches or beats it on four circuits (BV -66.7%, QAOA -7.4%; DJ and W-state at parity), while the embedding is more e-bit-efficient

on QPEexact (+32.7%) and VQE-SU2 (+11.1%), whose interaction graphs KaHyPar partitions almost perfectly. This suite is the one where the network model matters most: given the device's true data capacity `pytket-dqc` improves to a 10.6 geometric mean and the comparison reaches parity, and its remaining advantage on QPEexact turns out to require three times the communication capacity the device has. Section IV-C gives that analysis.

64q. The H-grid suite is where the gap from TeleSABRE is most consequential (Table IV). The suite comprises the six TeleSABRE-benchmark circuits plus the three class-completion additions (QPEexact, QAOA, Multiplier). dSABRE reduces gmean EPRs by **45.5%** over the seven circuits on which TeleSABRE converges, with per-circuit reductions ranging from −12.6% (QPEexact) to −78.9% (Graphstate), QFT/QNN at −50.2%/−62.8%, and the 13k-CX Multiplier at −31.2%. The per-core adaptive corner reservation (Section IV-F) supplies every core with escape slots, which resolves the earlier star-circuit weakness on GHZ: it now improves (9 vs. 11) rather than regressing, as no core is born fully packed and the local Δ-form score is no longer forced to chase the next CX out of a saturated core. Against pytket-dqc the comparison *reverses* on this suite: the 9-circuit geometric mean is +14.7% even after that compiler is given the device's own capacity (Section IV-C). Steiner-tree gate embedding reuses a few EPR pairs across many controlled operations, and it is markedly more e-bit-efficient on the structured circuits (QPEexact +129.7%, Graphstate +128.6%, AE +70.4%; GHZ and QFT at parity); dSABRE leads on the densest circuits (Multiplier −47.6%, Random −22.1%, QNN −16.0%), on which the embedding method does not scale or embed and a plain partitioner is used. Three things qualify the reversal, each examined below: the e-bit column charges no intra-core SWAPs, the two compilers start from different partitions, and—most consequentially here—seven of these nine distributions cannot be executed within the communication capacity the device provides at all (Section IV-C). On Random and QAOA, TeleSABRE fails to converge on any of three seeds; dSABRE completes both (721 and 529 EPR) under default scoring and proactive relief alone, without triggering checkpoint–rollback.

a) Shared-mapping comparison.: To isolate routing from allocation quality, we re-run dSABRE from each baseline's own initial layout. The TeleSABRE gap persists (−25% EPR on 25q, −43% on 64q from its `optimize_initial` layout); against `pytket-dqc`'s KaHyPar partition dSABRE wins on all three suites (−56.8% on 25q, −8.0% on 36q, −40.1% on 64q); the 36q margin is the narrowest because the sparse interaction graphs in that suite are already near-optimally partitioned by KaHyPar.

b) Entanglement-lifetime sensitivity.: Every `pytket-dqc` e-bit count above assumes an *unbounded* entanglement lifetime: the link qubit created by one starting process stays entangled until the last gate of its hyperedge has executed. Bandini *et al.* [18] argue that this is unrealistic—EPR-pair quality decays quickly, so the number of gates sharing one pair must be bounded—and cap gates-per-EPR at $D_{\max}$, proposing $D_{\max}$=3 as a realistic setting. We therefore re-score `pytket-dqc`'s distributions under this bound, using a counter that generalises its own cost model: an established link serves at most $D_{\max}$ gates before it dies and must be re-established from a still-live neighbour on the hyperedge's Steiner tree. The counter is conservative in `pytket-dqc`'s favour (intermediate links age only when they themselves serve gates; embedded and home-server gates age nothing; wall-clock time is ignored) and reproduces `pytket-dqc`'s own cost exactly at $D_{\max}=\infty$ on every distribution evaluated. dSABRE needs no adjustment at any $D_{\max}$: a teledata EPR is consumed instantaneously by its teleport, so teledata counts are lifetime-invariant by construction.

The +14.7% reversal of Table IV exists only above a lifetime threshold. Sweeping $D_{\max}$ over the 64q suite, the geometric-mean gap crosses parity between $D_{\max}$=10 and $D_{\max}$=5 and becomes a dSABRE win of **37.9**% at the realistic $D_{\max}$=3 (68.5% at $D_{\max}$=1). The circuits `pytket-dqc` wins under an unbounded lifetime are exactly those whose distributions hold links across many gates, and they inflate 2.7–4.8× at $D_{\max}$=3: QPEexact, its largest unbounded win, inflates 79→378 e-bits and flips to a 28.3% dSABRE advantage, and its winning QNN distribution requires individual link qubits to stay entangled across more than half the circuit's execution. Circuits whose hyperedges hold roughly one gate each (GHZ, Graphstate) are lifetime-insensitive, so those residual wins reflect partition quality rather than held entanglement, consistent with the shared-mapping result above. We conclude that gate teleportation's e-bit advantage is conditional on hardware sustaining entanglement across $\gtrsim$5–10 gates per pair, whereas teledata routing delivers its counts under any lifetime. The per-circuit sweep, and the same sweep on the smaller suites, are in the online appendices.

C. Matching pytket-dqc to the Physical Architecture

The e-bit counts of Table IV are the ones `pytket-dqc` reports on the network description it is normally given, and that description is more permissive than the device dSABRE is benchmarked on in two respects we can correct.

Communication capacity. `NISQNetwork` accepts a per-server communication capacity `server_ebit_mem`, the number of link qubits a core may hold *simultaneously*. Left unset—as in the standard usage—the constructor documents it as unbounded. dSABRE's architecture is not: a core has exactly one communication port per incident inter-core link, so core c can hold at most $\deg(c)$ link qubits at once, which is 2 or 3 on our architectures. *Data capacity.* The server sizes normally passed are an even split of the *logical* qubit count, unrelated to the device. Physically a core holds M qubits of which $\deg(c)$ are communication ports, leaving $M-\deg(c)$ data slots—14 on the B-grid, 13–14 on the H-grid, more room than the even split provides.

We therefore rebuild the network with data capacity $M-\deg(c)$ and communication capacity $\deg(c)$, and re-run the same distributor with the same seed budget. Table V reports the result. Two things follow, pulling in opposite directions.

First, the correction changes the numbers in both directions, and on the smaller suites it *helps* `pytket-dqc`, because

the physical data capacity is the more generous of the two: its geometric mean improves from 35.3 to 27.5 at 25 qubits and from 12.3 to 10.6 at 36. dSABRE's margin narrows from -56.6% to -44.3% at 25 qubits, and at 36 qubits the comparison crosses over to parity ($+1.8\%$, `pytket-dqc` marginally ahead) where the permissive model showed -12.3%. On the 64-qubit H-grid the same change goes the other way (168.0 to 176.3): the extra data capacity shifts the KaHyPar partition without paying for itself, and dSABRE's deficit narrows from $+20.0\%$ to $+14.7\%$. These are the figures Table IV now reports, in place of the earlier ones.

Second, the communication bound is not respected by the distributors at all. Setting `server_ebit_mem` leaves every reported cost unchanged, because `cost()` is computed from the hyperedge structure and the bound is checked only when the distribution is materialised into a circuit. Asking for that circuit is therefore a test of whether the reported answer is executable at all, and most of the time it is not: **13 of the 21 distributions cannot be built within the communication capacity the device provides**—four of six at 25 qubits, two of six at 36, and seven of nine at 64. The last column of Table V gives the smallest uniform capacity under which each can be built. AE needs at least 19 link qubits per core at 25 qubits and at least 18 at 64, against the 2 or 3 the device has. The circuits that do fit, at one link qubit each, are GHZ and Graphstate on both suites—precisely those whose hyperedges hold roughly one gate apiece.

The pattern is consistent: the distributions that need the most communication memory are the ones that amortise the most gates onto a single entangled state, which are exactly the ones on which gate teleportation looks cheapest. The 36-qubit suite makes this concrete. Under the corrected model `pytket-dqc`'s one clear win is QPEexact (52 e-bits against dSABRE's 65), and QPEexact's distribution requires 6 simultaneous link qubits per core; it cannot be executed on a device with 2. Its next-best relative showing, QAOA, needs more than 13. We do not claim this invalidates the gate-teleportation approach—it is a statement about a cost model, not about the physics—but a reader comparing the two columns should know that the e-bit counts and the EPR counts are not both achievable on the same machine.

This also connects the two corrections. Amortising many gates onto one entangled state is what makes gate teleportation cheap in e-bits, and it is simultaneously what forces a link qubit to be held *longer* (the lifetime assumption of Section IV-B) and to be held *alongside others* (the capacity assumption here). The three quantities are not independent knobs: a hyperedge spanning g gates buys a factor of g in e-bits, and pays for it in both link-qubit occupancy and required coherence. A device that cannot supply one generally cannot supply the other, which is why the lifetime sweep and the capacity audit single out the same circuits—AE, QFT, QNN, QPEexact—as the ones whose apparent advantage rests on the idealisation.

One route we could not take: `pytket-dqc` offers to repair a distribution to fit a bounded capacity (`to_pytket_circuit(allow_update=True)`), which would give a directly comparable constrained cost. On the 25-qubit QFT that call had not returned after nine minutes, against 9.2 s to produce the distribution in the first place, so we report realisability rather than a repaired count.

D. *Architecture Independence: Heavy-Hex Cores*

Section II-C models the architecture as an abstract weighted graph $G_{\mathcal{A}}$ queried only through three distance tables, but every experiment so far uses cores that are 2-D grids. If dSABRE's scoring or its capacity mechanism tacitly assumed grid structure, the generality claim would be empty. We test it by replacing the core with a topology that shares none of the grid's regularity.

Each core becomes a 27-qubit IBM heavy-hex tile of the Falcon family [19]: degrees range over $\{1, 2, 3\}$ rather than being uniform, six qubits are degree-1 leaves, and the tile's diameter is 12 against 6 for the 4×4 grid, so intra-core distances are roughly twice as long. Four such cores form a ring with one inter-core link per adjacent pair (108 physical qubits, 4 links). We run the six 64-qubit circuits common to every suite—dense (QNN, Random), structured (AE, QFT), and sparse (GHZ, Graphstate)—with the same hyperparameters of Table I and the same protocol; TeleSABRE runs on the same architecture through a generated device description. Nothing in the router or the layout is specialised for the new topology—the corner-reservation rule of Section IV-F is defined by vertex degree and remoteness, so on heavy-hex it selects leaf qubits instead of grid corners without modification.

Table VI reports the result. dSABRE's advantage over TeleSABRE is not merely preserved but slightly larger than on the H-grid (-48.2% against -45.5%), on a topology neither router was tuned for and with no parameter retuned—which is the claim Section II-C makes. TeleSABRE additionally fails to converge on QNN here, a circuit it completes on the H-grid, while dSABRE routes it. The capacity and congestion machinery transfers without adjustment in particular: the threshold τ and multiplier c_{pen} keep their defaults, and the fill-adaptive rule selects $k{=}4$ reserved leaves per core from the 59% fill on its own. Since these were the mechanisms whose architecture-dependence would most plausibly limit the model, their transfer is the part of this result we regard as load-bearing.

The absolute numbers move in an instructive way. EPR counts *fall* for both routers relative to the H-grid (dSABRE: AE $242 \to 87$, QFT $204 \to 126$, Random $721 \to 462$) while intra-core SWAP counts *rise* by 50–70%. This is the expected consequence of redistributing the same 64 logical qubits over four 27-qubit cores instead of six 16-qubit ones: a coarser partition leaves fewer gates crossing a core boundary, so fewer teleports are needed, but each core is now twelve hops across instead of six, so more local SWAPs are needed to bring operands together. Larger cores trade inter-core EPR pairs for intra-core SWAPs, and dSABRE follows that trade without retuning because both quantities enter its teleport score in their native units (Section III-D): d_{prep} counts actual SWAPs on the real core graph and c_{cap} reads actual free slots, so a change in core size or shape rescales both automatically.

E. Ablation Study

We ablate dSABRE's design choices along two axes: discrete mechanisms on/off, and continuous parameter sensitivity. All configurations inherit the defaults of Table I and report the *geometric-mean EPR count* (gmEPR) over the ablation subset defined in Table VII.

a) Mechanism ablation.: Table VII disables one component at a time on the 25q and 64q suites, by setting the corresponding parameter to zero, substituting the topological-order extended-set construction for the BFS-layer one, or disabling the mechanism entirely. The two suites stress different regimes: the 25q B-grid is sparsely loaded (qubits per core ≤ 7, ample headroom on every core), so congestion-driven mechanisms barely activate, while the 64q H-grid runs at $\sim$67% qubit utilisation and saturates inter-core ports under dense workloads.

The capacity penalty is the single most critical component on both suites: without it the router greedily teleports into full cores, triggering cascading evictions that inflate gmean EPR by $\sim$61% on 25q and $\sim$54% on 64q. This is the empirical core of the paper's design claim—the one term that encodes the architecture's finiteness is the one that matters most. The extended-set lookahead is second, contributing $+11\%$ on 25q and $+27\%$ on 64q; removing it collapses the scorer to a near-myopic policy, and its value grows with circuit size as dependency chains lengthen. Substituting a topological-order extended-set construction costs 6.8% on 25q and 8.6% on 64q as tabulated, and 10.9% on the 36q suite, which is not shown; the gain grows with the depth of the remaining DAG, as Proposition 1 predicts.

The hop-gain reward is the one term whose value is *scale-dependent*, and it carries a methodological warning. On this subset it looks marginal ($+1.8\%$ on 64q, exactly zero on 25q, where all twelve B-grid circuits tie to the EPR). Measured on the full 64-qubit suite, which includes the three largest circuits, removing it costs $+2.2\%$ overall and $+18.6\%$ on the 13 040-CX Multiplier alone (Section III-D). A term that is exactly inert on small instances and material on large ones is easy to dismiss as noise if only small instances are ablated—which is precisely what a six-circuit subset does. We report both bases rather than the more convenient one.

Congestion relief is near-neutral on both suites once the per-core adaptive layout is in place: it is inactive on 25q (0%) and mildly counterproductive on 64q (-8.7%, i.e. disabling it lowers gmean EPR). Because every core now begins with reserved escape capacity, the standing congestion the relief mechanism was designed to pre-empt is largely absorbed upstream by the layout; its residual value is confined to individual hard instances (e.g. 64q Random, where it enables completion) rather than the suite geometric mean.

b) Parameter sensitivity.: We sweep each continuous hyperparameter one-at-a-time on 25q and 64q. All defaults—w_e=0.25, c_{pen}=15, $|E|$=20, γ=0.9, w_{link}=10, and the relief bonus—sit on a flat region of their respective curves (within $\sim$ 2% of the local minimum), so performance is robust to small perturbations. The only sharp regressions occur when a control is pushed far from its default: e.g. $w_e \geq 1.0$ inflates 25q gmEPR by $3-5\times$ as the lookahead dominates immediate preparation costs, c_{pen}=0 doubles gmEPR by inviting deadlocks in saturated cores, and γ=1.0 (flat weighting) costs 7% on 64q. These observations confirm that the defaults are well-calibrated and that no single hyperparameter is a hidden lever; tuning beyond the defaults yields diminishing returns.

F. SabreLayout-Based Initial Layout

dSABRE bootstraps from Qiskit `SabreLayout` [11] run on the full architecture graph $G_{\mathcal{A}}$ (intra-core edges plus inter-core links), with two adaptations for the distributed setting.

Per-core corner reservation. SabreLayout packs each core to its full physical capacity. Two problems follow. First, an occupied communication port cannot accept an incoming teleport without first evicting its current occupant, so saturated cores either stall the inter-core scorer or force chains of relocations. Second, full cores trip the capacity penalty (Section III-D) on every candidate teleport into them, biasing the router away from the actually-preferred landing core. We therefore exclude, from the coupling map passed to SabreLayout, the k most-remote *corner* physical qubits of *every* core—the minimum-degree vertices farthest (by total shortest-path distance) from all other physical qubits, which in both B-grid and H-grid architectures sit furthest from any inter-core link—so that each core retains k free slots for eviction scratch space and stays below the capacity threshold. The count k is chosen adaptively as the largest value in $\{0, \ldots, 4\}$ that keeps the usable-slot fill $n_q/(N_{\text{cores}}(m^2-k))$ at or below 0.80; this yields k=4 on the 25/36/100/200-qubit suites and k=2 on the denser 64-qubit suite. A whole-chip variant that removes only the globally lowest-degree corners reserves slots in a single core under core-major qubit numbering, leaving the others fully packed; per-core reservation is what supplies every core with escape capacity.

Forward–backward–forward schedule. We run three routing passes (forward $\rightarrow$ reversed-DAG backward $\rightarrow$ forward) and keep the better of the two forward passes by EPR. The backward pass starts from the first forward pass's output layout ℓ_f and routes the reversed DAG, producing ℓ_b that pre-positions qubits near their early interaction partners; the third pass then warm-starts from ℓ_b. This is the same mechanism as SABRE's backward refinement [11] and outperforms pure-forward warm-restart. Best of three SabreLayout seeds is reported.

G. Compile Time

The online appendices tabulate wall-clock compilation time on the 64-qubit suite.

Times scale with CX count: TeleSABRE spans 0.02 s (GHZ) to 86 s (Multiplier, 13 040 CX); dSABRE spans 0.05 s to 462 s. dSABRE is slower on most circuits where both converge. The gap is an implementation constant, not an asymptotic one, and three pieces of evidence separate the two. First, the measured growth matches the complexity of Eq. 7 rather than exceeding it: dSABRE's time grows with CX count at the same rate as TeleSABRE's, and the ratio between them stays bounded—within about one order of magnitude—across a $200\times$ range of circuit size, which is the signature

of a per-operation overhead rather than a worse algorithm. Second, the two implementations differ in exactly the way that constant predicts: TeleSABRE is a compiled C++ binary with cached distance lookups, while dSABRE is pure Python calling NetworkX for shortest paths inside the routing loop, and interpreted-versus-compiled overheads of one order of magnitude are the norm for this kind of inner loop. Third, the ordering inverts whenever the constant stops dominating: on Graphstate dSABRE is 78× *faster* (0.05 s vs. 3.92 s), because TeleSABRE's three-success sampling spends many iterations before three runs converge there. Nothing in Eq. 7 is specific to Python, so a compiled implementation would be expected to close most of the gap; we regard that as engineering rather than algorithm design and have not pursued it. In absolute terms the cost is modest against what it buys: dSABRE routes the largest circuit in the suite in under eight minutes and roughly halves its EPR count, and EPR pairs—not compile seconds—are the scarce resource on the hardware this targets.

H. Large-Circuit Scalability

To stress-test dSABRE substantially beyond the main evaluation range we run dSABRE, TeleSABRE, and `pytket-dqc` on three additional suites using the MQT-Bench QFT circuit (IBM native-gate, Qiskit opt3). The architectures scale the H-grid of the 64q suite:

- **100q**: H-grid 2×3 5×5 (150 physical), six 25-qubit cores.
- **200q**: H-grid 4×3 5×5 (300 physical), twelve 25-qubit cores.
- **360q**: H-grid 2×3 9×9 (486 physical), six *81*-qubit cores.

Two properties of this sweep should be stated before its numbers are read, because both bear on how comparable the three rows are.

The QFT circuits are not all the same construction. The 25- and 64-qubit QFTs used in Table IV's main panels are MQT-Bench v1.1.0 circuits, in which the two operands of a controlled phase may be arbitrarily far apart (interaction range $n-1$). The 100-, 200- and 360-qubit circuits come from the current MQT-Bench release, whose QFT is a *banded* approximation: only qubit pairs within distance 19 interact, which we confirmed by regenerating all three and matching gate count, depth and interaction-pair count exactly. Their interaction graphs are the 19th power of a path, with $19n-190$ pairs. Both are legitimate approximate QFTs, but they are different circuit families and we do not present the five sizes as one series.

The 360-qubit row is an easier problem than the 200-qubit one, despite being larger. With a fixed interaction band of 19, what decides how much traffic crosses a core boundary is the number of logical qubits per core. At 100 and 200 qubits that is 17, below the band, so most gates straddle a boundary; at 360 qubits it is 60, three times the band, so only gates near a boundary cross. Counting directly, 56% and 57% of gates are inherently inter-core at 100 and 200 qubits against 16.5% at 360. The 360-qubit row therefore presents fewer than half as many cross-core gates as the 200-qubit row (1,900 against 4,270) while containing 84% more gates in total, and every router's EPR count falls accordingly. The clean scaling pair is 100q→200q, which holds the band and the core size fixed and doubles the core count; 360q is best read as a separate large-core regime. TeleSABRE is best-of-3-seed with a 600 s per-seed timeout for 200q+; dSABRE uses its default configuration; `pytket-dqc` uses HypergraphPartitioning (best of 5 seeds). Each router runs from its own initial layout, so the Δ_{tket} column should be read with this layout asymmetry in mind in addition to the e-bit counting caveat (Section IV-A).[1]

Table IV reports the three large-QFT points. dSABRE beats TeleSABRE at both sizes where the latter converges (−10.1% at 100q, −43.6% at 360q), and at *200q* TeleSABRE fails within the 600 s timeout while dSABRE completes (834 EPR); the checkpoint–rollback escape does not fire on the best-EPR run but salvages one of the three SabreLayout seeds that would otherwise abort. Against `pytket-dqc` the picture is the opposite and consistent across all three sizes: QFT is a deep controlled-phase chain, the circuit family its Steiner-tree embedding targets most directly, and it wins on raw e-bits at every size. Giving it the device's own capacity (Section IV-C) narrows every margin—+95.6% to +62.8% at 100q, +86.6% to +82.9% at 200q, and +221% to +119.6% at 360q, where the correction nearly doubles its e-bit count—but does not close them. The two caveats of Section IV-A bear on the column as well: it charges no intra-core SWAPs, and each router starts from its own layout. Controlling for the second reverses the result at all three sizes (footnote below). At these sizes COVEREMBEDDINGSTEINERDETACHED does not complete within its budget, so the cheaper PARTITIONINGHETEROGENEOUS distributor is reported throughout this panel. Compile times for 360q QFT are 281 s (TeleSABRE, C++, best of 3 seeds) and, for dSABRE (Python, SabreLayout + fwd→bwd→fwd), of order 700 s per SabreLayout seed; the Python overhead is the primary driver of dSABRE's longer wall time relative to the compiled C++ TeleSABRE. pytket-dqc's strongest distributor is the most expensive of the three—COVEREMBEDDINGSTEINERDETACHED exceeds a 25-minute budget at this size (its cheaper HYPERGRAPHPARTITIONING partitioner runs in ≈550 s).

I. Cost Analysis

The combined-cost ratio c_{tele}:c_{swap} is hardware-dependent: near-term microwave-coupled modular superconducting qubits keep teleportation within a small constant factor of local SWAPs [1], while photonic-link architectures have measured EPR-generation times three to four orders of magnitude slower than local gates [9], [10], pushing the ratio towards 100:1 or beyond. A sweep in the online appendices varies c_{tele} from 10 to 100 with $c_{\text{swap}} = 3$ held fixed, reporting geometric-mean cost reduction of dSABRE vs. TeleSABRE.

The pattern is consistent across all three suites: the gap to TeleSABRE widens as c_{tele} grows. At the teleport-friendly

[1]For a layout-controlled point of comparison, re-running dSABRE from `pytket-dqc`'s HypergraphPartitioning layout gives EPR counts of 280 (100q), 1014 (200q), and 563 (360q), versus `pytket-dqc`'s 815, 1124, and 669 — corresponding to Δ_{tket} of −65.6%, −9.8%, and −15.8%. In this matched-layout setting dSABRE beats `pytket-dqc` at all three sizes, most narrowly at 200q where its SabreLayout configuration is marginally worse.

end (c_{tele}=10, ratio $c_{\text{tele}}/c_{\text{swap}} \approx 3.3$) the cost model penalises teleports least, so TeleSABRE's higher EPR count is discounted; dSABRE still saves 20.6% on 25q, 39.0% on 36q, and 23.8% on 64q. At c_{tele}=100 (ratio $\approx$ 33) — the SWAP-favourable regime of near-term photonic-link hardware [9], [10] — dSABRE's smaller EPR count dominates and the savings reach 35.2%, 43.2%, and 38.9% respectively. Across the entire practically relevant range dSABRE dominates TeleSABRE, and the advantage grows with EPR cost — exactly the regime where the cost model becomes stricter.

V. Related Work

The Introduction already places dSABRE within the partition–communicate–schedule decomposition of the DQC compilation stack and lists the major lines of work in each stage. Here we focus on what specifically distinguishes dSABRE from the closest prior art and on the single-QPU SABRE family that we build on but do not subsume.

SABRE-style distributed routers. TeleSABRE [15] is our closest prior work and direct experimental baseline. Both routers follow the SABRE loop, but differ in how they score inter-core moves: TeleSABRE evaluates candidates via a *contracted communication-graph energy* that averages per-gate Dijkstra distances over the whole front layer, whereas dSABRE uses a local, gate-centric score (Eq. 6) evaluated only on the gates the moving qubit touches (Section III-D).

Three further differences are worth stating precisely, because they are where the measured gap comes from. *Selection structure.* TeleSABRE places SWAP, teleport, and telegate candidates in one pool, scores each by the energy of the layout it would produce, and takes a single arg min; a flat bonus is then subtracted from the two communication primitives. With the default bonus of 100 against energies of order ten, that arg min reduces in practice to *teleport whenever a feasible teleport exists*. dSABRE makes the priority explicit and inverts it: intra-core front gates are resolved by SWAP first, and teleport candidates are scored only when no front gate has both operands in one core. *Capacity.* Both routers carry a capacity signal inside the energy, but of different shape. TeleSABRE adds a fixed penalty to the communication qubits of any core with at most two free slots, charged along every core its Dijkstra path traverses; dSABRE charges $c_{\text{pen}} \max(0, \tau - f_{\text{dst}})$, graded in the free capacity of the immediate landing core. Both additionally filter infeasible landings. The ablation attributes more of dSABRE's gap to this term than to any other component (Section IV-E), which we read as evidence that the graded form is what matters, not the presence of a capacity signal as such. *Lookahead weighting.* TeleSABRE discounts the g-th gate it emits by $(1 + g/10)$, an emission-index weight that does not distinguish gates at different DAG depths within a slice; dSABRE discounts by BFS depth, $\gamma^{\text{dep}(g)}$ (Proposition 1). Finally, on stalls TeleSABRE relies on a safety valve that narrows the front to a single gate and disables the extended set, with no progress argument, where dSABRE's checkpoint–rollback escape bounds the cost of escaping a stall (Proposition 2). TeleSABRE supports telegate candidates while dSABRE currently uses teledata only.

DMapS [14] is a concurrent end-to-end DQC compiler that pairs a KaHyPar partitioner with a SABRE-derived router and shares our joint EPR-plus-local-SWAP objective. Two design choices set it apart: its only cross-core primitive is a two-EPR *remote SWAP* (whereas dSABRE uses one-way teledata into a layout-reserved free slot), and its initial allocation comes from a KaHyPar partition of the interaction graph. On dense circuits dSABRE consumes substantially fewer EPRs—2–4× on 25q QFT/QNN/Random and 1.6–1.8× on 64q—while DMapS wins on maximally sparse circuits where KaHyPar partitions almost perfectly. A code-level audit, the per-circuit head-to-head, and a controlled fill-ratio sweep showing parity once the device fill ratio approaches 1 are in the online appendices.

Allocation methods built on hypergraph partitioning [3], [4] are complementary to dSABRE: any can supply the initial layout its routing loop consumes. Similarly, time-aware schedulers [7], [8] sit upstream or downstream of the routing stage dSABRE addresses.

Single-QPU qubit mapping. On a single-core processor the routing problem reduces to inserting intra-chip SWAPs. The closest single-QPU antecedent to our work is the front+extended-set scorer SABRE [11] that we and TeleSABRE both build on. Babu et al. [20] use auxiliary-qubit gate teleportation on a heavy-hex IBM Eagle processor, achieving 10–25% depth reduction versus standard SABRE; the setting is single-QPU and not directly comparable to multi-core routing.

VI. Future Work

Composing dSABRE with burst execution. dSABRE optimises EPR count on a fixed circuit DAG. Burst-execution passes such as AutoComm [21] and its collective extension CollComm [22] operate earlier in the toolchain: they use gate commutativity to reorder consecutive non-local gates on the same wire (or across multiple wires) and amortise the whole block onto a single shared entanglement state, reporting EPR savings of 50–75% on amenable circuits; the lifetime-bounded gate grouping of Bandini *et al.* [18] tempers such savings with the $D_{\max}$ constraint we adopt in Section IV-B. The two approaches operate at different layers and should compose well: a burst-extraction pre-pass would expose longer wire-aligned gate sequences, and dSABRE's BFS extended set would let the teleport score capture their full benefit. Quantifying the joint improvement—especially on circuits where a burst's beneficiary core is itself a function of the routing decisions dSABRE makes—is an open question, and recent integrations of gate-teleportation amortisation with SABRE-style mappers [20] suggest the interaction is non-trivial.

Parallel teleportation for circuit-runtime minimisation. dSABRE optimises EPR count and emits one teleport per iteration; wall-clock runtime depends additionally on whether teleports with disjoint links and staging sets can fire concurrently. A natural follow-on is a scheduling pass over dSABRE's teleport trace that batches non-conflicting moves, co-designed with denser inter-core links [23] and entanglement buffer qubits [22], [24]. Time-sliced compilers [7], [8] provide a starting point but are not paired with a SABRE-style scoring loop; naively adding links without retuning the routing energy can *increase* EPR consumption.

Beyond EPR count: latency and fidelity. This paper optimises a resource count, not an execution time or an output fidelity. Two of the three system-level costs are already partly visible: intra-core SWAP count, the dominant contributor to routed depth, is reported beside EPR count in every table and falls alongside it, and entanglement quality enters through the lifetime bound of Section IV-B, since capping gates-per-EPR is how a decaying link appears in a cost model. What remains open is a joint objective. EPR count, depth and per-operation error rates are not interchangeable—a teleport is slow and noisy, a SWAP fast and cheap, and the exchange rate is hardware-specific—so a fidelity-aware router needs a device error model and a scheduling pass in the loop, both outside the routing stage dSABRE addresses. The cost-ratio sweep is a first step: the conclusions are stable across the two orders of magnitude of relative teleport cost that separate microwave-coupled from photonic-link hardware.

VII. Conclusion

We presented dSABRE, a SABRE-style router for multi-core distributed quantum computers that, on each iteration of a lookahead-driven loop, resolves intra-core front-layer gates by SWAP scoring and only scores inter-core teleport candidates when the intra-core front is empty, with both phases sharing a common decay-weighted lookahead form. The empirical takeaway is narrower than we first expected and correspondingly firmer. Two scoring choices account for most of the gap to the state of the art: an explicit capacity penalty that keeps the scorer out of saturated cores, which contributes more than any other single component, and a BFS-layer inter-core extended set that respects DAG dependencies. Proactive congestion relief and the checkpoint–rollback escape, which we had expected to carry part of the gap, turn out to be neutral on the geometric mean once every core is given reserved escape capacity; their value is that dSABRE completes circuits on which the state of the art does not converge. We report them on those terms. More broadly, the results indicate that distributed routing benefits less from a richer per-candidate scoring formula than from ensuring that the architectural capacity signal reaches the scorer in *graded* form—scaled to how full the landing core already is, rather than acting only as a feasibility filter or a fixed-threshold step (Section III-A).

Artefact availability.: The dSABRE implementation, the routed-circuit benchmark suites, per-circuit JSON results, and the online appendices referenced throughout this paper are available at https://github.com/ebony72/dsabre.

Acknowledgments

This work was partially supported by the Australian Research Council (Grant No. DP250102952 and DP220102059). The author used Anthropic's Claude (via Claude Code) to assist with: (i) refactoring portions of the dSABRE implementation, (ii) authoring benchmark scripts and aggregating per-circuit JSON results into the tables of Section IV, (iii) drafting the LaTeX sources for the architecture and teleport-candidate figures (Figures 2 and 4), and (iv) copy-editing prose throughout the manuscript. All AI-generated content was reviewed, verified, and corrected by the author, who takes full responsibility for the correctness and originality of the work.

References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] P. Andres-Martinez and C. Heunen, "Automated distribution of quantum circuits via hypergraph partitioning," *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.

[4] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[7] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.

[8] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.

[9] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[10] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[11] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[12] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2024.

[13] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "LightSABRE: A lightweight and enhanced SABRE algorithm," 2024. [Online]. Available: https://arxiv.org/abs/2409.08368

[14] T. Luo, Y. Zheng, Y. Deng, and X. Fu, "DMapS: End-to-end qubit mapping and routing for distributed quantum computing architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025, code: https://github.com/RoccoLoter/DMapS.

[15] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[16] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, "Elementary gates for quantum computation," *Physical Review A*, vol. 52, no. 5, pp. 3457–3467, 1995.

[17] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[18] M. Bandini, D. Ferrari, S. Carretta, and M. Amoretti, "Optimized compilation for distributed quantum computing," 2026, arXiv:2602.24062.

[19] C. Chamberland, G. Zhu, T. J. Yoder, J. B. Hertzberg, and A. W. Cross, "Topological and subsystem codes on low-degree graphs with flag qubits," *Physical Review X*, vol. 10, no. 1, p. 011022, 2020.

[20] A. P. Babu, O. Kerppo, A. Muñoz Moller, M. Haghparast, and M. Silveri, "Gate teleportation-assisted routing for quantum algorithms," 2025.

[21] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[22] A. Wu, Y. Ding, and A. Li, "CollComm: Enabling efficient collective quantum communication based on EPR buffering," 2022.
[23] P. Escofet, S. B. Rached, S. Rodrigo, C. G. Almudever, E. Alarcón, and S. Abadal, "Interconnect fabrics for multi-core quantum processors: A context analysis," in *Proceedings of the 16th International Workshop on Network on Chip Architectures (NoCArc)*, 2023, arXiv:2309.07313.
[24] J. Liu, A. Zang, M. Suchara, T. Zhong, and P. D. Hovland, "Hardware-software co-design for distributed quantum computing," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–6.

Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.

TABLE V

PYTKET-DQC ON THE NETWORK IT IS NORMALLY GIVEN (*published*: LOGICAL-SPLIT DATA CAPACITY, UNBOUNDED COMMUNICATION MEMORY) VERSUS ONE MATCHING DSABRE'S DEVICE (*physical*: DATA CAPACITY $M - \deg(c)$, COMMUNICATION CAPACITY $\deg(c)$). DSABRE EPR COUNTS ARE FROM TABLE IV. *ports needed* IS THE SMALLEST UNIFORM COMMUNICATION CAPACITY UNDER WHICH THE PUBLISHED DISTRIBUTION CAN BE MATERIALISED, AGAINST THE 2 THE DEVICE PROVIDES. THE SEARCH IS BUDGET-LIMITED ON THE LARGER CIRCUITS, SO $>k$ DENOTES A LOWER BOUND: IT WAS STILL FAILING AT k WHEN THE BUDGET EXPIRED. EITHER WAY A VALUE ABOVE 2 MEANS THE DISTRIBUTION CANNOT BE EXECUTED ON THIS DEVICE, AND THOSE ROWS ARE MARKED †; *n/d* MARKS THE ONE CASE WHERE NEITHER THE SCAN NOR THE PER-DISTRIBUTION CHECK RETURNED WITHIN ITS BUDGET, SO NO VERDICT IS CLAIMED FOR THAT ROW.

	`pytket-dqc` e-bits		dSABRE	ports
Circuit	published	physical	EPR	needed
25 logical qubits, B-grid, 2 ports per core				
ae†	48	48	23	19
ghz	3	2	1	1
graphstate	4	2	2	1
qft†	61	57	33	10
qnn†	115	127	48	12
random†	480	314	178	7
gmean	35.3	27.5	15.3	
36 logical qubits, same B-grid				
bv	3	1	1	1
dj	3	2	3	1
qaoa†	148	144	137	>13
qpeexact†	49	52	65	6
vqe_su2	9	12	10	1
wstate	6	8	6	1
gmean	12.3	10.6	10.8	
64 logical qubits, H-grid, 2–3 ports per core				
ae†	141	142	242	>18
ghz	5	9	9	1
graphstate	9	7	16	1
qft†	183	204	204	>7
qnn†	657	605	508	>6
random†	948	925	721	>4
qpeexact†	97	118	271	>5
qaoa†	513	473	529	>7
multiplier†	2956	2891	1514	n/d
gmean	168.0	176.3	202.2	
QFT scalability sweep				
100q†	114	137	223	>7
200q†	447	456	834	>2
360q	155	285	626	n/d

TABLE VI

THE 64-QUBIT SUITE ON A NON-GRID ARCHITECTURE: A RING OF FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL QUBITS, 4 INTER-CORE LINKS). SAME CIRCUITS, HYPERPARAMETERS, AND PROTOCOL AS TABLE IV, RESTRICTED TO THE SIX CIRCUITS COMMON TO EVERY SUITE; ONLY THE ARCHITECTURE DIFFERS. *TELESABRE FAILS TO CONVERGE.

		TeleSABRE		dSABRE		ΔEPR
Circuit	CX	EPR	SWAP	EPR	SWAP	(%)
ae	1962	124	2538	87	2256	−29.8
ghz	63	5	235	2	43	−60.0
graphstate	64	46	538	12	151	−73.9
qft	1966	154	2562	126	2656	−18.2
qnn*	8126	—	—	239	9499	—
random	1627	740	7585	462	6006	−37.6
gmean (5)		79.9	1442	41.4	748	**−48.2**

TABLE VII

MECHANISM ABLATION ON THE 25Q AND 64Q SUITES. ONE COMPONENT IS DISABLED PER ROW; ALL OTHERS REMAIN AT THEIR DEFAULTS. SAME PROTOCOL AS THE MAIN RESULTS. TO KEEP THE GEOMETRIC MEANS COMPARABLE ACROSS CONFIGURATIONS, BOTH SUITES ARE ABLATED ON THEIR SIX COMMON CIRCUITS (AE, GHZ, GRAPHSTATE, QFT, QNN, RANDOM), SO THE **FULL** 64Q BASELINE IS THE SIX-CIRCUIT MEAN AND DIFFERS FROM THE SEVEN- AND NINE-CIRCUIT MEANS OF TABLE IV. ALL ROWS SHARE THIS BASIS; THE HOP-GAIN TERM ADDITIONALLY ACTS OUTSIDE IT, AND IS QUANTIFIED ON THE FULL SUITE IN SECTION III-D.

	25q		64q	
Configuration	gmEPR	Δ (%)	gmEPR	Δ (%)
Full (baseline)	15.3	—	117.3	—
No capacity penalty ($c_{\text{pen}} = 0$)	24.7	+60.8	180.0	+53.5
No extended-set lookahead ($w_e = 0$)	17.1	+11.3	148.3	+26.5
Topological extended set (not BFS)	16.4	+6.8	127.4	+8.6
No congestion relief	15.3	0.0	107.1	−8.7
No hop-gain reward ($w_h = 0$)	15.3	0.0	119.5	+1.8